

Atelier 4 : Développement et Déploiement d'une Application Web Sécurisée avec JWT

1. Introduction

Dans cet atelier, nous avons développé une application web complète permettant l'authentification sécurisée via JSON Web Token (JWT) et la gestion d'une liste de membres.

L'application est composée de :

- Une API REST développée avec Express
- Une interface utilisateur développée avec React (Vite)
- Une sécurisation des routes via JWT
- Un déploiement en ligne via Vercel

L'objectif principal était de comprendre le fonctionnement de l'authentification par token et de relier un frontend React à une API sécurisée.

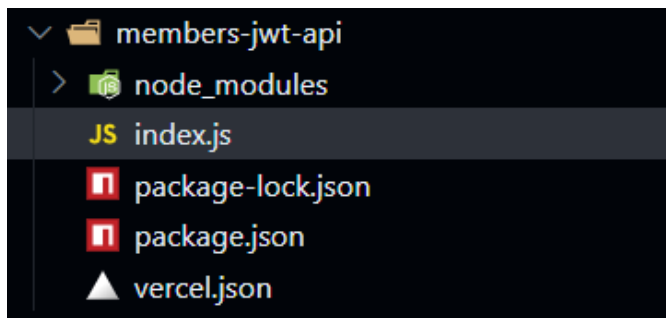
2. Développement du Backend (API Express)

Le backend a été développé avec :

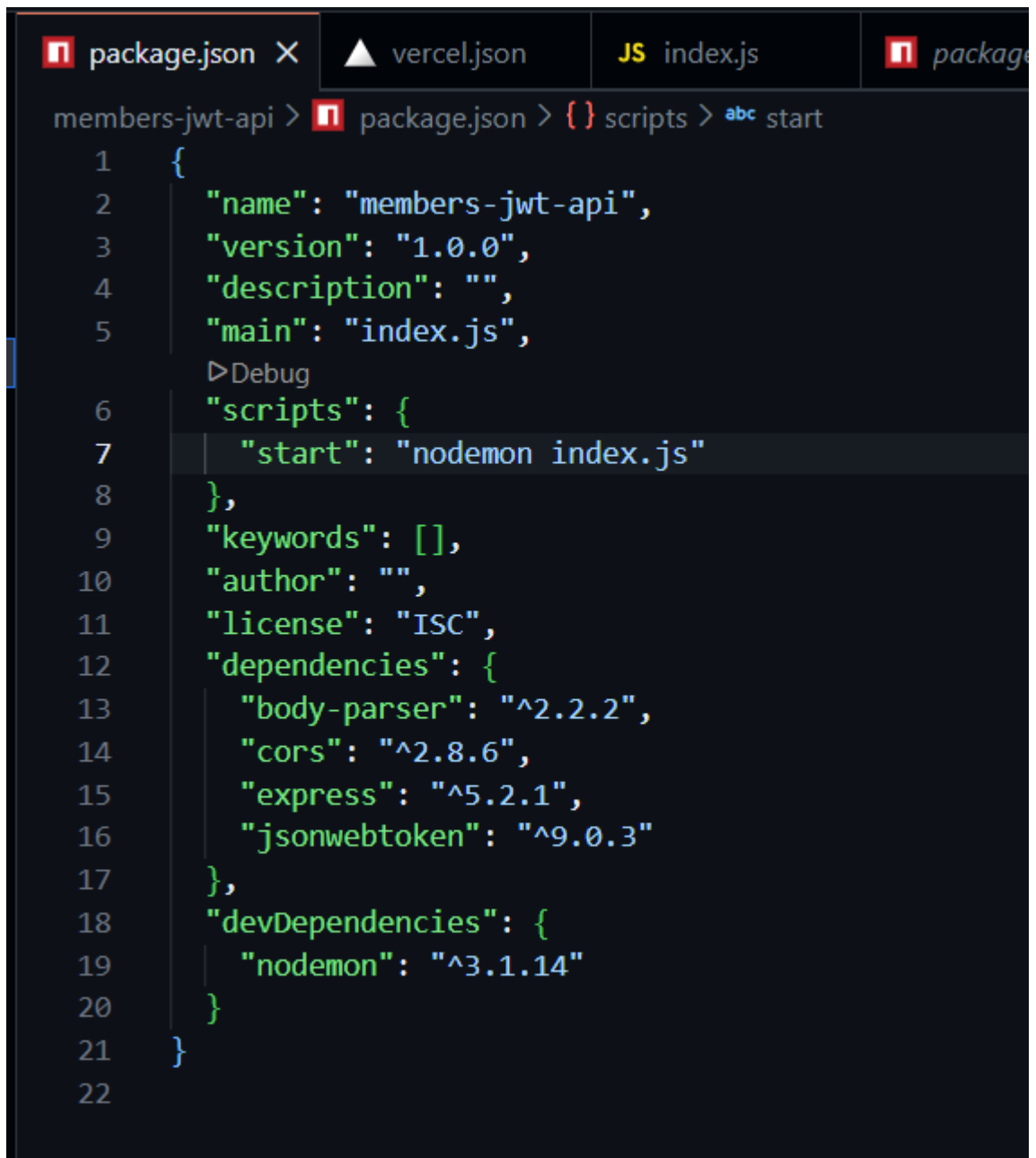
- Node.js
- Express
- jsonwebtoken
- Middleware personnalisé pour la vérification du token

Le serveur expose plusieurs routes sous le préfixe :

/api



```
JS index.js X JS api.js Login.jsx Members.jsx App.jsx ...
members-jwt-api > JS index.js > app.use() callback
1  const express = require("express")
2  const jwt = require("jsonwebtoken")
3
4  const app = express()
5
6  /* ===== CORS FIX FOR VERCEL ===== */
7
8  app.use((req, res, next) => {
9    res.header(
10      "Access-Control-Allow-Origin",
11      "https://atelier4-jwt-frontend.vercel.app"
12    )
13    res.header(
14      "Access-Control-Allow-Methods",
15      "GET, POST, PUT, DELETE, OPTIONS"
16    )
17    res.header(
18      "Access-Control-Allow-Headers",
19      "Content-Type, Authorization"
20    )
21
22    // Handle preflight requests
23    if (req.method === "OPTIONS") {
24      return res.sendStatus(200)
25    }
26
27    next()
28  })
29
30 /* ===== BODY PARSER ===== */
31
32 app.use(express.json())
33 app.use(express.urlencoded({ extended: true }))
34
35 /* ===== ENV CONFIG ===== */
36
37 const SECRET = process.env.JWT_SECRET || "members secret key"
```



```
members-jwt-api > package.json > {} scripts > abc start
1  {
2    "name": "members-jwt-api",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "start": "nodemon index.js"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "dependencies": {
13     "body-parser": "^2.2.2",
14     "cors": "^2.8.6",
15     "express": "^5.2.1",
16     "jsonwebtoken": "^9.0.3"
17   },
18   "devDependencies": {
19     "nodemon": "^3.1.14"
20   }
21 }
22
```

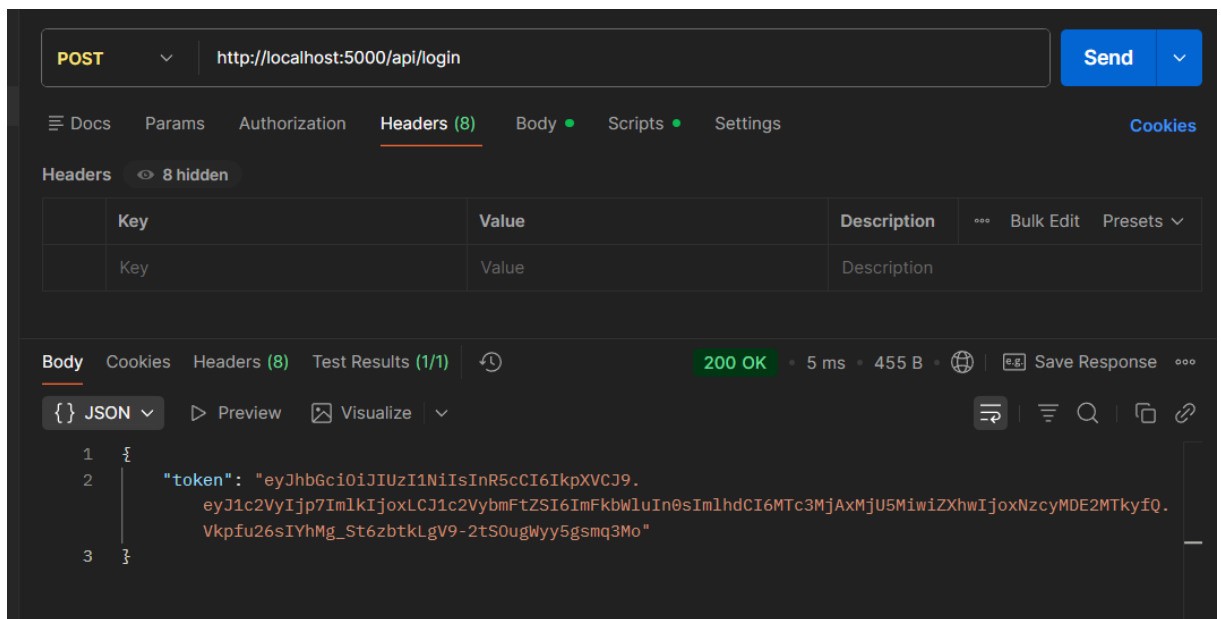
2.2 Authentification avec JWT

Une route POST /api/login a été créée.

Fonctionnement :

1. L'utilisateur envoie username et password
2. Vérification des identifiants
3. Génération d'un token JWT valide 1 heure

4. Envoi du token dans la réponse



```
41  const testUser = {
42    id: 1,
43    username: "msbenyaishe",
44    password: "said2005"
45  }
```

2.3 Middleware de Vérification du Token

Un middleware verifyToken a été mis en place.

Il :

- Vérifie la présence du header Authorization
- Extrait le token
- Vérifie sa validité
- Autorise ou refuse l'accès

Les routes protégées sont :

- GET /api/members
- POST /api/members
- PUT /api/members/:id

- DELETE /api/members/:id

```

80  /* ===== VERIFY TOKEN MIDDLEWARE ===== */
81
82  function verifyToken(req, res, next) {
83    const authHeader = req.headers.authorization || ""
84
85    if (!authHeader.startsWith("Bearer ")) {
86      return res.status(403).json({
87        message: "Access denied. Token required."
88      })
89    }
90
91    const token = authHeader.split(" ")[1]
92
93    jwt.verify(token, SECRET, (err, decoded) => {
94      if (err) {
95        return res.status(401).json({
96          message: "Invalid or expired token."
97        })
98      }
99
100     req.user = decoded.user
101     next()
102   })
103 }

```

2.4 Gestion CRUD des membres

Fonctionnalités :

- Affichage de tous les membres
- Ajout d'un membre
- Modification d'un membre
- Suppression d'un membre

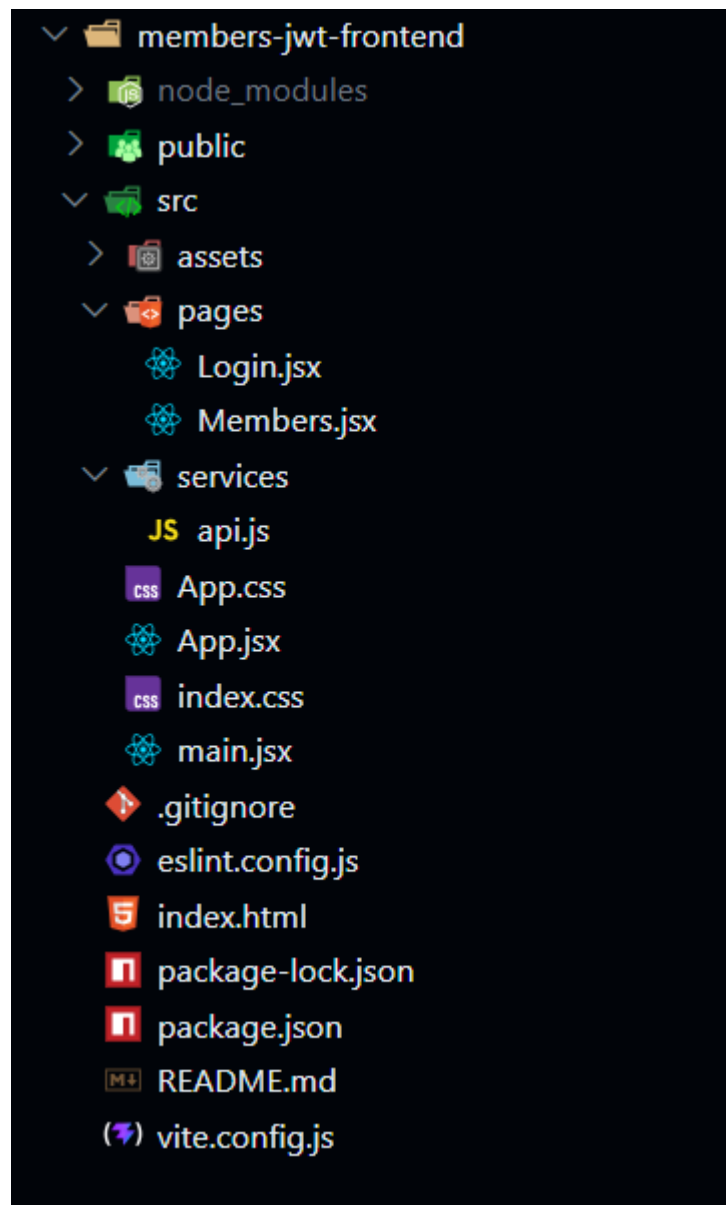
Les données sont stockées dans un tableau simulant une base de données.

3. Développement du Frontend (React avec Vite)

Le frontend a été créé avec Vite pour une meilleure performance.

Structure principale :

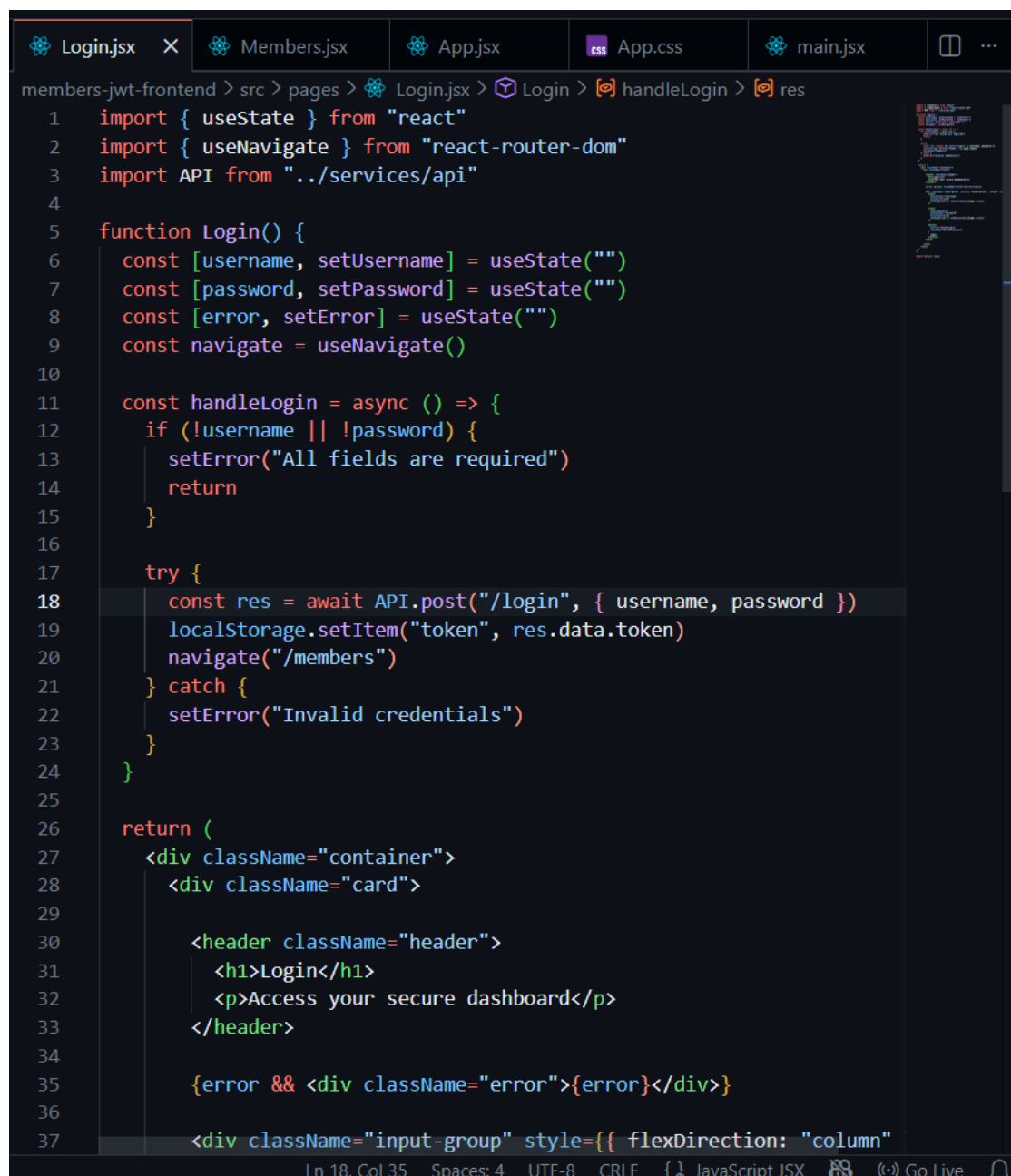
- Login.jsx
- Members.jsx
- services/api.js
- React Router



3.2 Page de Connexion

La page Login permet :

- Saisie du username
- Saisie du password
- Envoi d'une requête POST vers /api/login
- Récupération du token
- Stockage du token dans localStorage
- Redirection vers la page Members



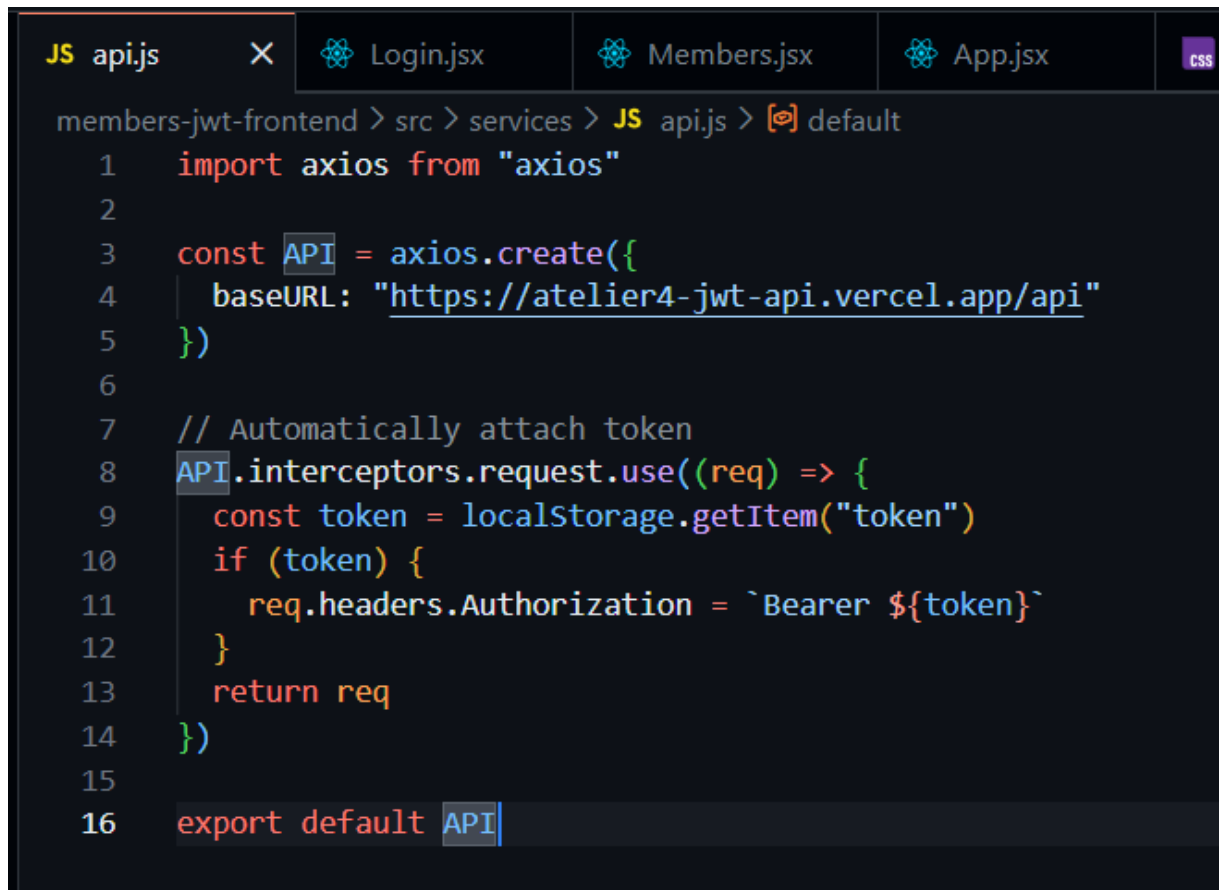
```
members-jwt-frontend > src > pages > Login.js > Login > handleLogin > res
1  import { useState } from "react"
2  import { useNavigate } from "react-router-dom"
3  import API from "../services/api"
4
5  function Login() {
6    const [username, setUsername] = useState("")
7    const [password, setPassword] = useState("")
8    const [error, setError] = useState("")
9    const navigate = useNavigate()
10
11    const handleLogin = async () => {
12      if (!username || !password) {
13        setError("All fields are required")
14        return
15      }
16
17      try {
18        const res = await API.post("/login", { username, password })
19        localStorage.setItem("token", res.data.token)
20        navigate("/members")
21      } catch {
22        setError("Invalid credentials")
23      }
24    }
25
26    return (
27      <div className="container">
28        <div className="card">
29
30          <header className="header">
31            <h1>Login</h1>
32            <p>Access your secure dashboard</p>
33          </header>
34
35          {error && <div className="error">{error}</div>}
36
37          <div className="input-group" style={{ flexDirection: "column"
Ln 18, Col 35  Spaces: 4  UTF-8  CRLF  {} JavaScript JSX  Go Live
```

3.3 Configuration Axios et Interceptor

Un fichier api.js a été configuré avec Axios.

Un interceptor a été ajouté pour :

- Récupérer automatiquement le token depuis localStorage
- L'ajouter dans le header Authorization



```
JS api.js X Login.jsx Members.jsx App.jsx CSS
members-jwt-frontend > src > services > JS api.js > default
1  import axios from "axios"
2
3  const API = axios.create({
4    |   baseURL: "https://atelier4-jwt-api.vercel.app/api"
5  })
6
7  // Automatically attach token
8  API.interceptors.request.use((req) => {
9    |   const token = localStorage.getItem("token")
10   |   if (token) {
11     |     req.headers.Authorization = `Bearer ${token}`
12   |   }
13   |   return req
14   | }
15 )
16 export default API
```

3.4 Page Members (CRUD)

La page Members permet :

- Affichage des membres
- Ajout d'un membre
- Modification d'un membre
- Suppression d'un membre
- Déconnexion (suppression du token)



```
1 import { useEffect, useState } from "react"
2 import { useNavigate } from "react-router-dom"
3 import API from "../services/api"
4
5 function Members() {
6   const [members, setMembers] = useState([])
7   const [name, setName] = useState("")
8   const [editId, setEditId] = useState(null)
9   const [errorMsg, setErrorMsg] = useState("")
10  const navigate = useNavigate()
11
12  /* ===== FETCH MEMBERS ===== */
13
14  const fetchMembers = async () => {
15    try {
16      const res = await API.get("/members")
17      setMembers(res.data.result || [])
18    } catch (err) {
19      // If token invalid or expired → logout
20      localStorage.removeItem("token")
21      navigate("/")
22    }
23  }
24
25  useEffect(() => {
26    fetchMembers()
27  }, [])
28
29  /* ===== CREATE / UPDATE ===== */
30
31  const handleSubmit = async () => {
32    if (!name.trim()) {
33      setErrorMsg("Name is required")
34      return
35    }
36
37    try {
```

4. Gestion du CORS

Pour permettre la communication entre le frontend et le backend hébergés sur des domaines différents, une configuration CORS a été mise en place.

Les origines autorisées :

- <http://localhost:5173>
- <https://atelier4-jwt-frontend.vercel.app/>

Les headers autorisés :

- Content-Type
- Authorization

```
6  /* ===== CORS FIX FOR VERCEL ===== */
7
8  app.use((req, res, next) => {
9    res.header(
10     "Access-Control-Allow-Origin",
11     "https://atelier4-jwt-frontend.vercel.app"
12   )
13   res.header(
14     "Access-Control-Allow-Methods",
15     "GET, POST, PUT, DELETE, OPTIONS"
16   )
17   res.header(
18     "Access-Control-Allow-Headers",
19     "Content-Type, Authorization"
20   )
21
22   // Handle preflight requests
23   if (req.method === "OPTIONS") {
24     return res.sendStatus(200)
25   }
26
27   next()
28 })
```